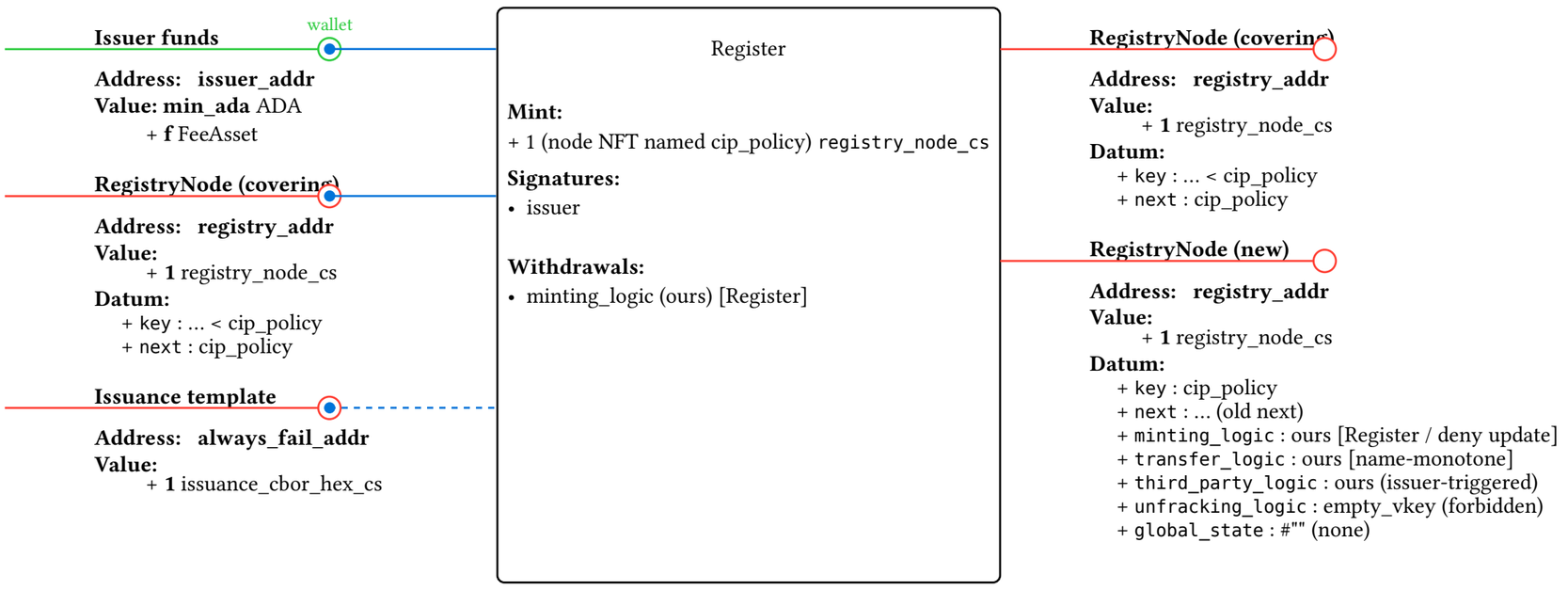
Contents

1 - Tokenized bond — register (T0) 2
2 - Tokenized bond — deposit (unit) 3
3 - Tokenized bond — transfer (T1) 4
4 - Tokenized bond — authorize claims (self-transfer) 5
5 - Tokenized bond — coupon claim (step k) 6
6 - Tokenized bond — issuer claim (step k, third party) 7
7 - Tokenized bond — redemption (burn-to-pay) 8
8 - Tokenized bond — graduation (T4) 9
9 - Tokenized bond — issuer graduation (T4, third party) 10

1 - Tokenized bond — register (T0)

The issuer inserts the instrument's RegistryNode into the CIP-113 registry: the insertion spends the covering node, requires the issuance template and creates the new node — no certificate is minted here, so the same register serves every later deposit.



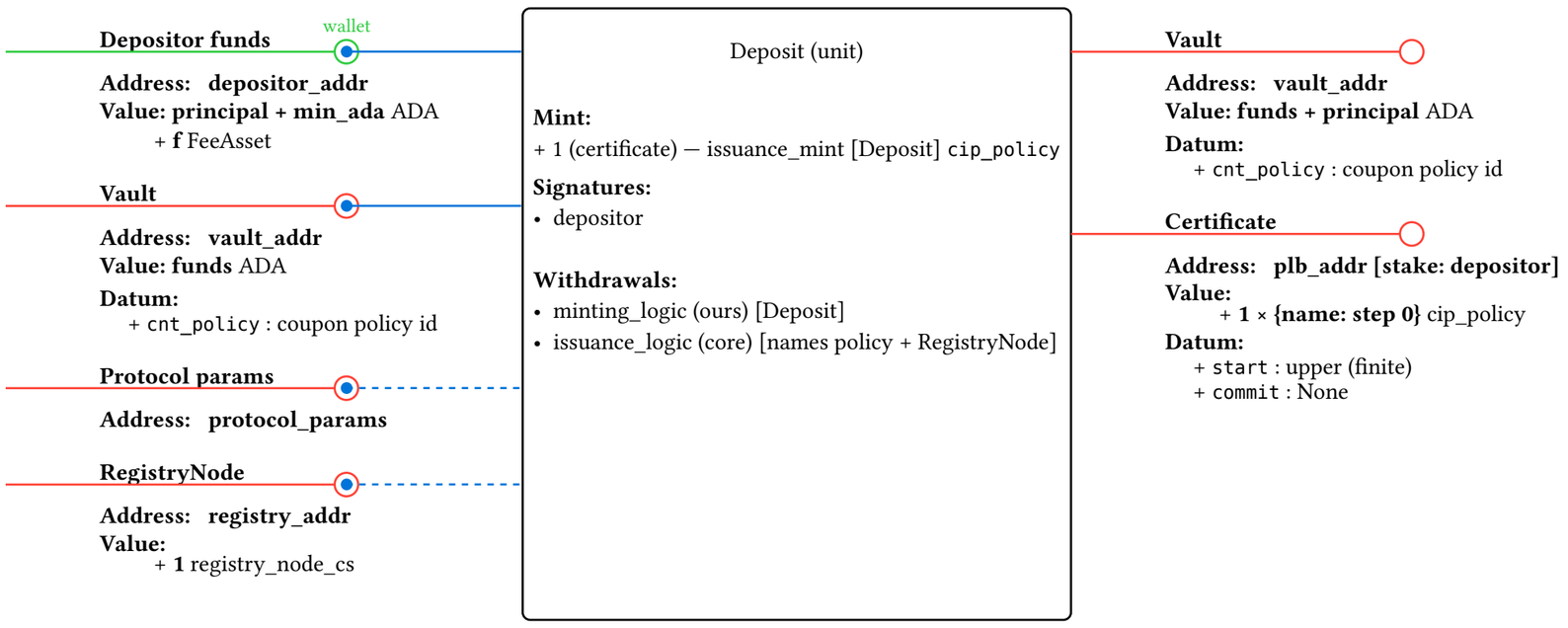
Notes:

- The core `registry` mint handler inserts the node: it requires the one-shot `IssuanceCborHex` **template** as a reference input (locked at `always_fail`) to bind `cip_policy` to our minting logic, spends the covering node, mints exactly one node NFT named `cip_policy`, and emits the two node outputs with sorted `key` / `next`.
- Our minting logic runs once as `[Register]`: it authorises the issuer, asserts `tx.mint` carries no `cip_policy` entries (register-only — deposits come later), and refuses registry-node updates, so the stance recorded in the new node is frozen for the instrument's life.
- No token is minted, so `issuance_mint` never runs and the core `issuance_logic` withdraw-0 is not needed here. The ADA vault is assumed created and funded by the owner (company/country) outside this design — see spec §7.

Figure 1: Register: the issuer inserts the node once; deposits can follow

2 - Tokenized bond — deposit (unit)

Anyone deposits one bond unit: the principal ADA moves into the vault and the same transaction mints that depositor's CIP-113 certificate, recording its timeline origin and the step-0 name.



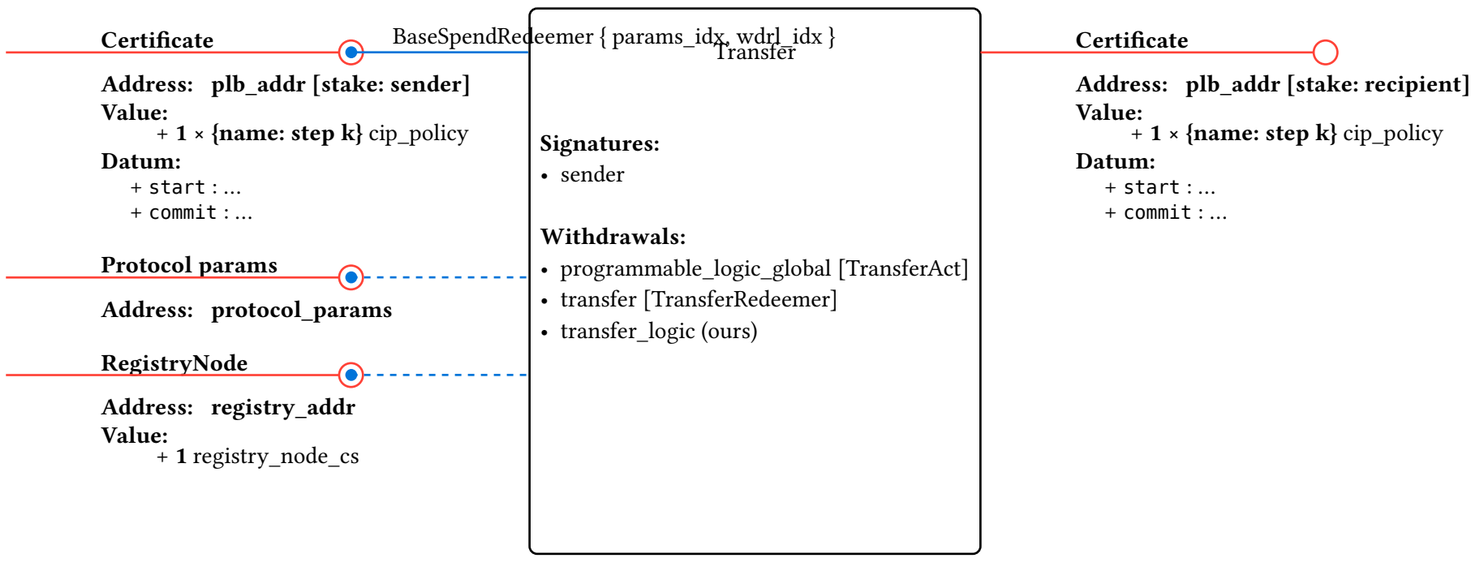
Notes:

- The deposit is permissionless and per unit: the minting logic admits the certificate only when the vault's net ADA gain equals exactly the unit `principal`, so the certificate and its backing exist atomically (I6). No authority signature is needed — the depositor signs only to spend their own funds.
- The `Deposit` mode pins `start` to the transaction's validity upper bound (finite); since the ledger guarantees `upper ≥ now`, the timeline can never be backdated — it can only start later, at the depositor's own cost. The certificate is minted with the step-`0` name and `commit: None`.
- The certificate is the instrument's only CIP-113 token; exactly one unit is minted per deposit (no certificate inputs, no merge), always the step-`0` name. Units deposited at different times run independent timelines from their own `start`.
- Coupons are plain native assets minted later by the coupon policy; nothing else is minted here.

Figure 2: Deposit: one unit of principal into the vault, one certificate to the depositor

3 - Tokenized bond — transfer (T1)

The holder moves the certificate freely; the transfer logic only enforces that the claim state never rewinds — its `step` stays the same.

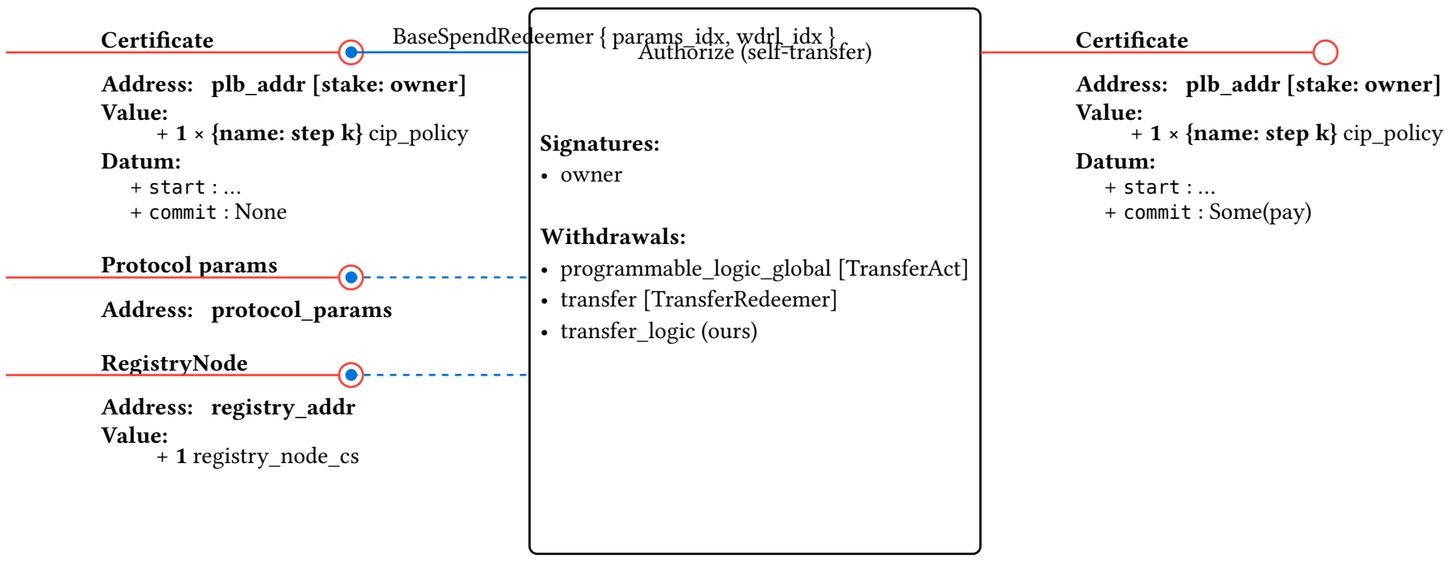


Notes:
- Ownership rides the certificate's inline stake credential: moving the token moves the claim on that unit's remaining coupons and principal, and the datum (`start`, `commit`) and the step name are carried across unchanged.
- The transfer logic resolves the governed `cip_policy` from the RegistryNode and enforces name monotonicity (I7): with a continuation, the step name and `start` are preserved, and `commit` may change only when the stake credential does not (the owner's self-transfer, next page); on an owner change it must be `None`.
- At most one certificate per transaction, quantity exactly one in and out (no merge or split). The coupon claim page is the admitted name advance (step `k - 1 → k`, burn+mint); retirement is a `cip_policy` burn on the graduation page.

Figure 3: Transfer: the certificate moves, the claim state never rewinds

4 - Tokenized bond — authorize claims (self-transfer)

The owner opts in to issuer triggering — or revokes it — by re-outputting the certificate to themselves with a committed payment credential.

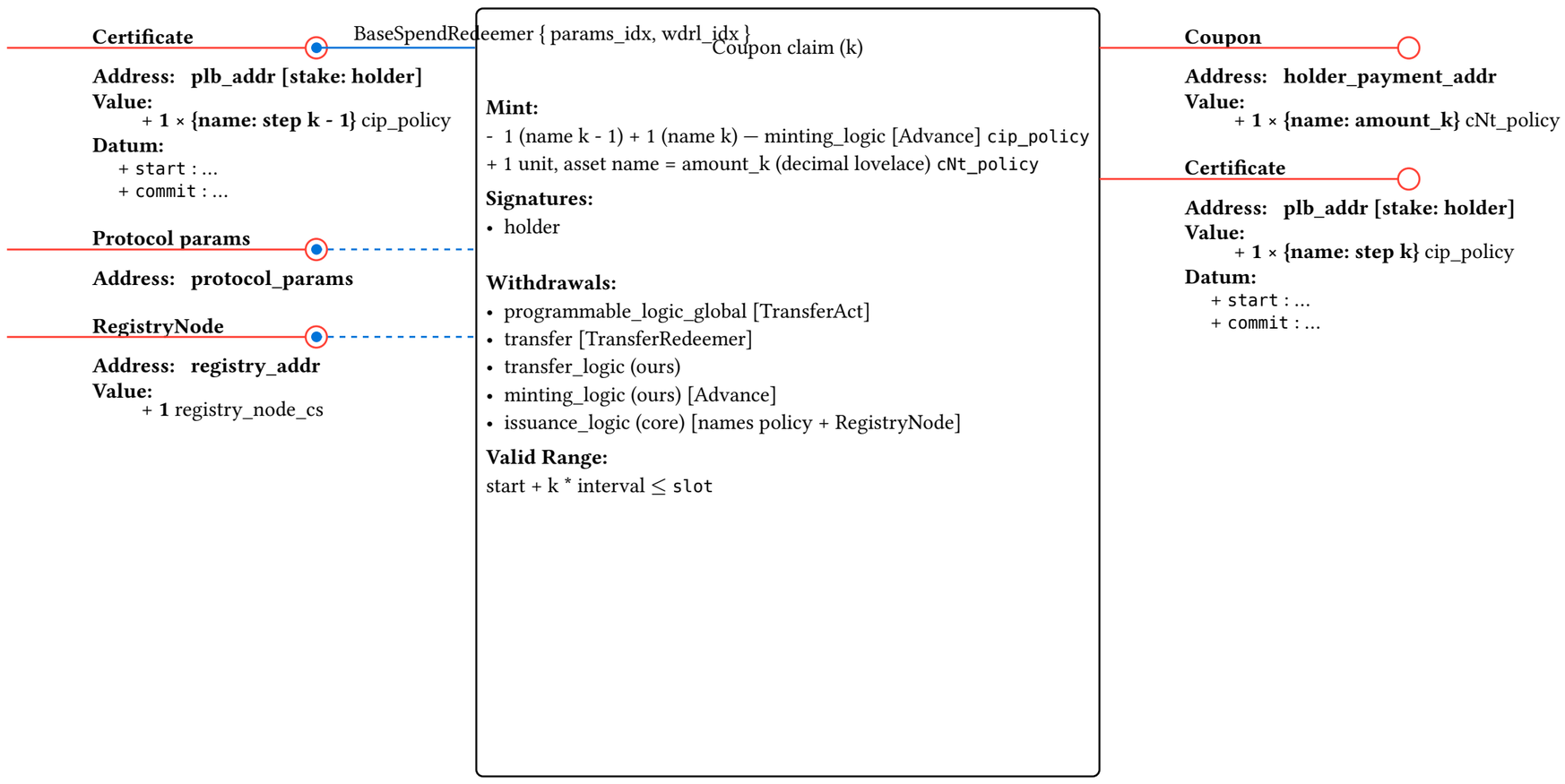


Notes:
- The owner writes `commit: Some(pay)` (or `None` to revoke) on a self-transfer: same stake in and out, step name and `start` preserved. Only the owner can authorise it, so the commitment is authentic.
- `commit` is cleared automatically when the owner changes — a transfer to a different stake must set it to `None` — so the issuer can never pay a previous owner.
- With the commitment in place the issuer may trigger this certificate's coupon claims and graduation through the third-party path (issuer pages), always paying `pay`; the owner keeps the self-service path.

Figure 4: Authorize: the owner commits a payment credential on a self-transfer

5 - Tokenized bond — coupon claim (step k)

Once `now ≥ start + k * interval` the holder claims the step's coupon. The claim burns the certificate's name `k - 1` and mints the name `k` one (plus the coupon), so no step can be claimed twice.

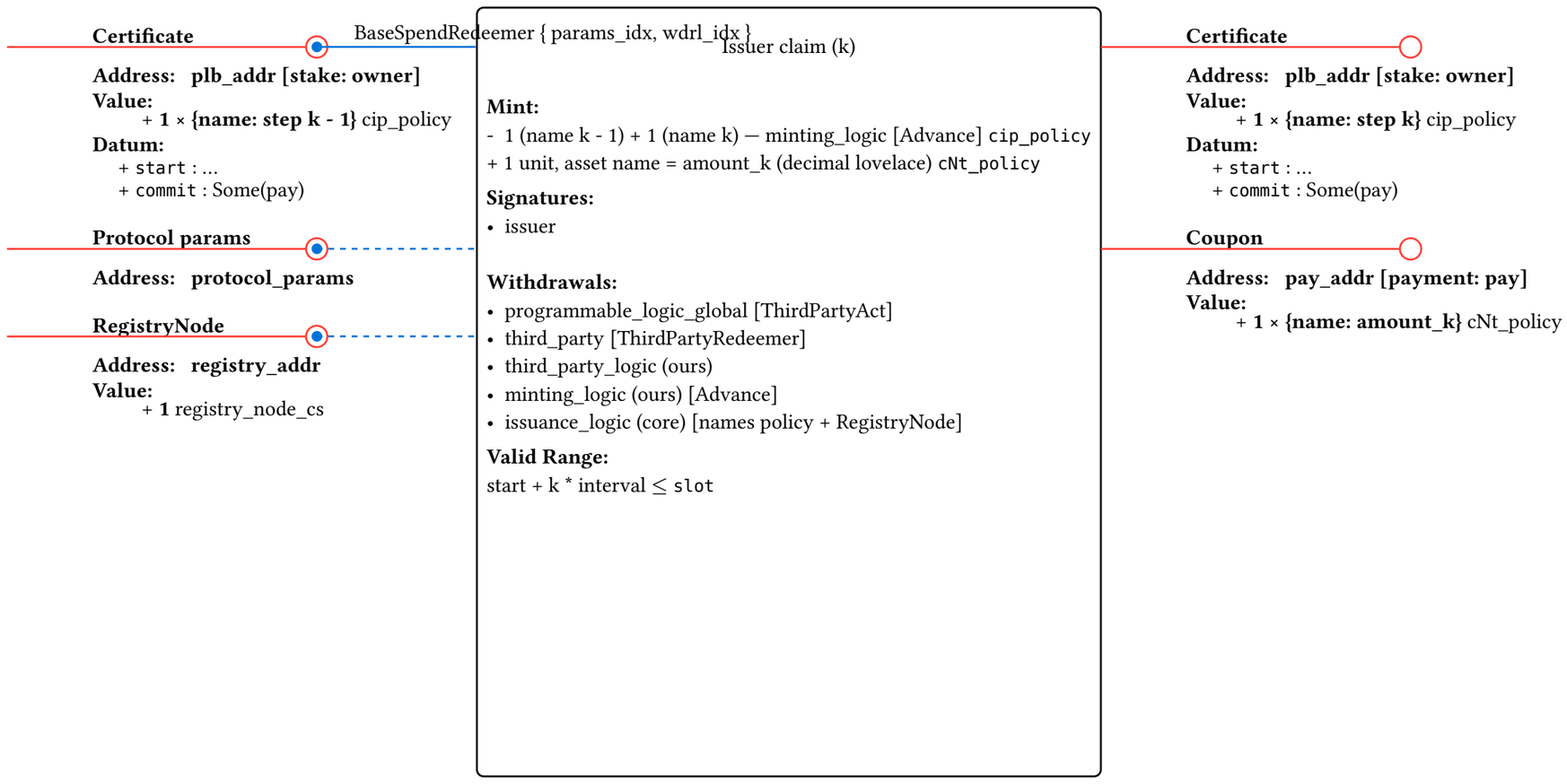


Notes:
- The minting logic's `Advance` mode burns exactly the spent name `k - 1` and mints exactly the name `k` (quantity `1` each), with the datum (`start`, `commit`) preserved — the certificate advances in place.
- The coupon policy admits the coupon mint only when the validity range reaches the certificate's baked `start + k * interval`, the spent name is `k - 1` and the continuation's name is `k`. That anchor makes each step claimable exactly once per certificate.
- The coupon is a native asset whose **name is its value**: name `"1124"` is worth 1124 lovelace. Two steps of equal amount share a name and are fungible on purpose.
- The claim is an owner-signed PLB spend of the certificate; it never touches the vault. When the certificate commits a payment credential, the issuer can claim the same step through the third-party path (next page).

Figure 5: Coupon claim at step k (holder-signed, step-anchored)

6 - Tokenized bond — issuer claim (step k, third party)

When the certificate commits a payment credential, the issuer claims the step's coupon on the owner's behalf through the third-party path. The paired output is the next certificate itself — no ghost shell.

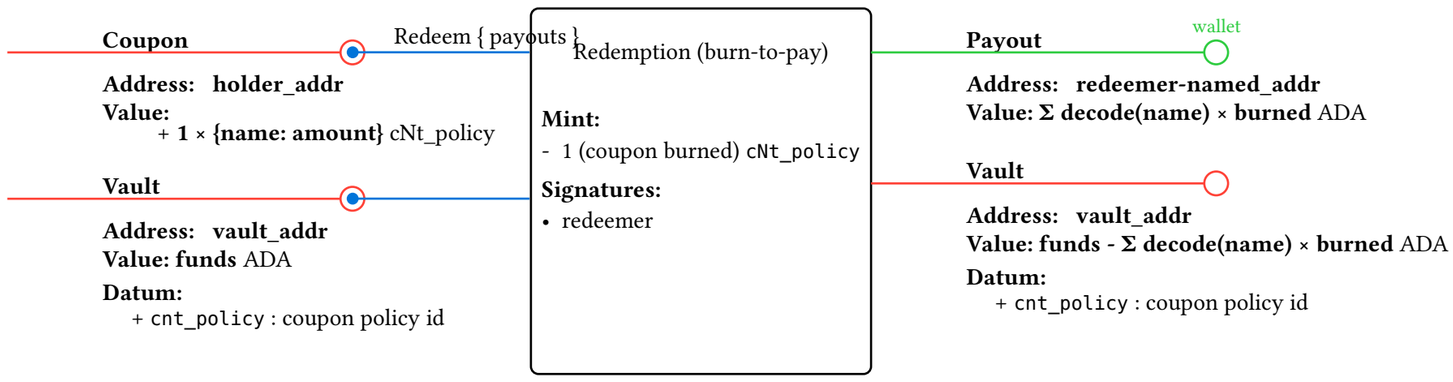


Notes:
- The third-party path pins the paired output's address and `datum` byte-for-byte, but **not the subject policy's asset name**: the pair can carry the name `k` certificate in place, so an issuer-claimed coupon leaves no certificate-less shell.
- The minting logic's `Advance` mode burns exactly the spent name `k - 1` and mints exactly the name `k` (quantity `1` each); the burn and mint balance under the base layer's aggregate conservation rail. `third_party_logic` requires the issuer's signature and `commit = Some(pay)`, validates the exact `k - 1 → k` transition and binds the coupon to `pay`. No owner signature is needed.
- The gate is unchanged — `now ≥ start + k * interval`, and the coupon policy still binds the mint to the name advance — so the issuer can trigger but never claim early or redirect.

Figure 6: Issuer claim: third-party spend, the pair is the next certificate

7 - Tokenized bond — redemption (burn-to-pay)

A coupon is redeemed by burning it at the vault: the vault validator pays out exactly the ADA the burned coupons name, and nothing else leaves.

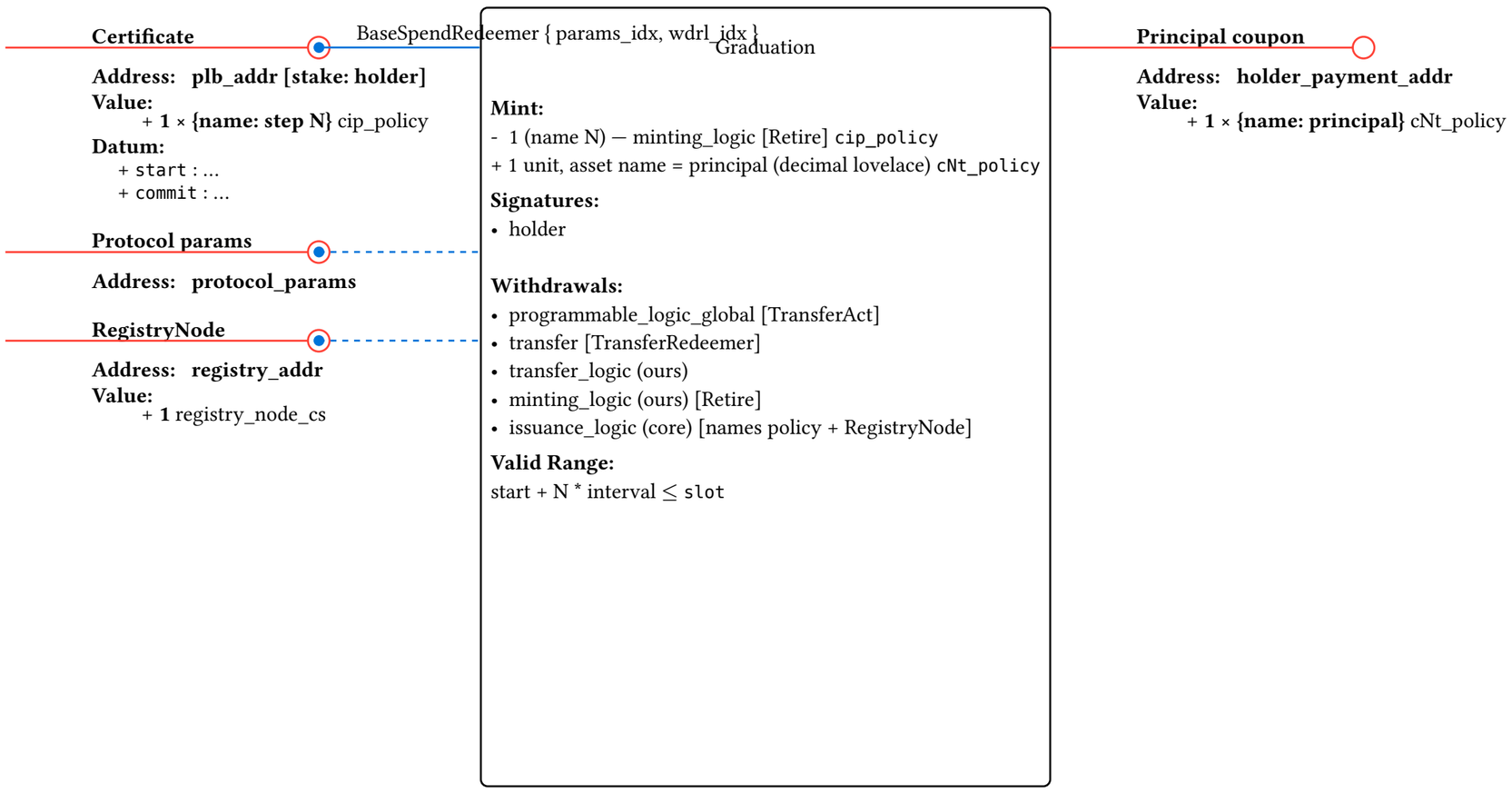


Notes:
- The vault validator sums `decode(name) × burned_quantity` over the negative coupon mints and requires the vault's net ADA loss to equal that sum exactly. A redeemer cannot ask for more than the coupons are worth, and the vault cannot short-pay.
- The payout goes to the address(es) the redeemer names in the same transaction.
- Redemption burns the coupon, so no coupon is ever paid twice.
- No owner signature is needed on this path; the burn-to-pay rule is enforced on-chain.

Figure 7: Redemption: burn coupons, the vault pays exactly what they name

8 - Tokenized bond — graduation (T4)

From the unit's maturity (`start + N * interval`) on, the holder claims the principal-only coupon and retires the certificate. The principal is the deployment's baked unit amount.

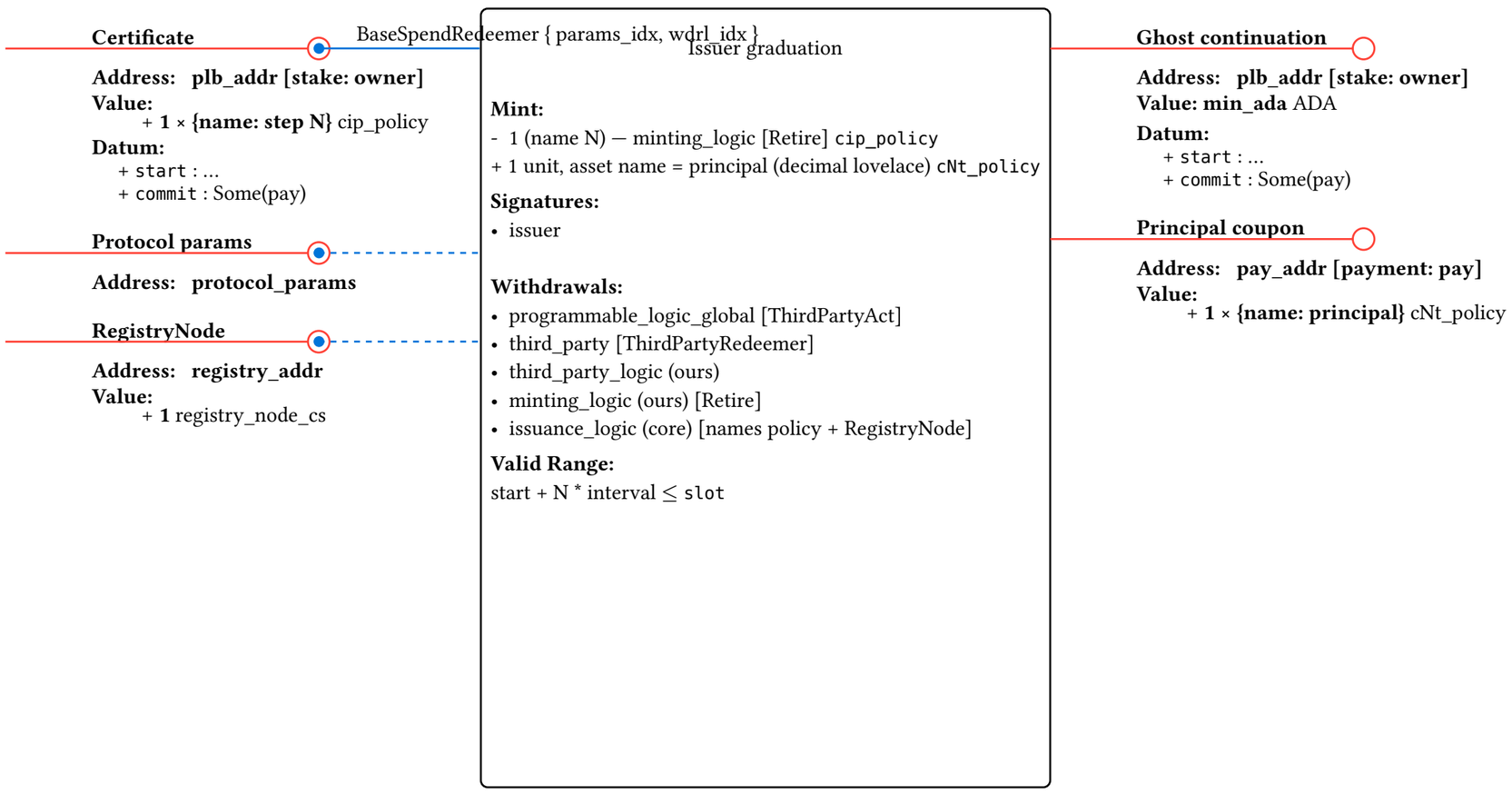


Notes:
- The principal is the deployment's baked unit amount, so the graduation cannot over- or under-claim.
- The certificate is **retired** by a `cip_policy` burn of name `N`, on or after its `start + N * interval` maturity — our minting logic's `Retire` mode validates the burn shape and payout, and the transfer logic admits no continuation on this shape only.
- The principal coupon is then redeemed at the vault like any other coupon (see the redemption page): burn it, the vault pays the principal. When the certificate commits a payment credential, the issuer can retire it through the third-party path (next page).

Figure 8: Graduation: claim the principal-only coupon, retire the certificate

9 - Tokenized bond — issuer graduation (T4, third party)

When committed, the issuer retires the certificate on the owner's behalf: the certificate is burned and the principal coupon is minted to the commitment.



Notes:

- The burn rides the issuance chain (`minting_logic [Retire]` + core `issuance_mint`); the core `issuance_logic` delegates the burn's custody to the third-party path, whose aggregate rail accounts for the negative mint. No owner signature is needed.
- `third_party_logic` requires the issuer's signature and `commit = Some(pay)`, validates name `N`, and binds the principal coupon to `pay`. Because the certificate disappears here, the paired continuation stays as a certificate-less ghost shell — the design's only shell.
- Timing is unchanged (`now ≥ start + N * interval`), so the issuer can only retire at maturity; the owner keeps the self-service graduation of the previous page. The shell is inert and the owner can sweep it later with an owner-signed transfer-path spend.

Figure 9: Issuer graduation: third-party burn, principal bound to the commitment